illustrated in the following listing, by passing in a name for the constraint and the matching type that should be used.

Listing 4.6 Custom route constraints registered using `RouteOptions`

```
builder.Services.AddRazorPages();
builder.Services.Configure<RouteOptions>(options =>
{
    options.ConstraintMap.Add("city", typeof(CityRouteConstraint));    ❶
});
```

❶ The constraint is registered with the name "city".

Once the constraint has been registered, it can be used just like any other by placing the name of the constraint after the parameter name:

```
@page "{cityName:city}"
```

4.2.6 Creating additional routes

When you customize the route template through the `@page` directive, you affect the routing system's resulting template. In addition to replacing the default route template, you can specify additional templates a page can match. The most direct way to do this is to use the `AddPageRoute` method when configuring `RazorPagesOptions` (which is similar to specifying an alternative root folder for pages), as shown in listing 4.7, where the Index page has an additional route applied to it, so it can be found at the URL /FindMe as well as the routes the system generates.

Listing 4.7 Adding additional route templates via configuration

```
builder.Services
    .AddRazorPages()
    .AddRazorPagesOptions(options =>
    {
        options.Conventions.AddPageRoute("/Index", "FindMe");
    });
```